Banking Platform Database Architecture

Table of Contents

Overview

Banking is the canonical use case where PostgreSQL's ACID guarantees are not a nice-to-have but a hard requirement. This schema implements double-entry bookkeeping (every debit has a corresponding credit), an immutable ledger, real-time fraud detection tables, and regulatory reporting structures. The design prioritizes correctness above all else: no money is ever created or destroyed; every cent is traceable.

Requirements

Functional Requirements

- Customer accounts: checking, savings, credit
- Double-entry ledger: every transaction creates two journal entries (debit/credit)
- Transfer between accounts (same bank and inter-bank via ACH/SWIFT)
- Deposits and withdrawals
- Interest calculation for savings and credit
- Card management (debit/credit)
- Standing orders and direct debits
- Statements and account history
- Fraud detection: velocity checks, anomaly flagging
- Regulatory reporting: AML (Anti-Money Laundering), KYC (Know Your Customer)
- Audit trail: every data change is logged

Non-Functional Requirements

- 10M+ customers
- 200M+ accounts
- 500M+ transactions per year (~1,600/second average, ~10K/second peak)
- Zero tolerance for data loss (synchronous replication, fsync=on)
- ACID compliance at SERIALIZABLE isolation for fund transfers
- 7-year data retention for regulatory compliance
- Encryption at rest and in transit

- PCI-DSS compliance for card data

Capacity Estimation

Metric	Estimate
Customers	10M
Accounts	25M (avg 2.5 per customer)
Transactions per year	500M
Transactions per second (peak)	10K
Ledger entries per year	1B (2 per transaction)
Fraud alerts per day	50K
ACH batches per day	10M items

Storage Estimates

Table	Rows (7yr)	Row Size	Total Storage
customers	10M	1KB	10GB
accounts	25M	500B	12.5GB
transactions	3.5B	600B	2.1TB
ledger_entries	7B	300B	2.1TB
audit_log	10B	400B	4TB
fraud_alerts	130M	500B	65GB
statements	25M × 84mo = 2.1B	200B	420GB

Total regulated storage: ~10TB (hot), ~100TB with archival
Write throughput required: 10K TPS × 2 ledger entries + audit = ~30K rows/second at peak

---

Schema Design

```
-- ============================================================
-- EXTENSIONS
-- ============================================================
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";      -- for encrypted PII storage

-- ============================================================
-- ENUM TYPES
-- ============================================================
CREATE TYPE customer_status AS ENUM ('prospect', 'active', 'restricted', 'closed',
```

```
'deceased');
CREATE TYPE kyc_status AS ENUM ('not_started', 'in_review', 'approved', 'rejected',
'expired');
CREATE TYPE account_type AS ENUM ('checking', 'savings', 'credit', 'loan',
'fixed_deposit', 'investment');
CREATE TYPE account_status AS ENUM ('active', 'dormant', 'frozen', 'closed');
CREATE TYPE currency AS ENUM ('USD', 'EUR', 'GBP', 'INR', 'JPY', 'CAD', 'AUD');
CREATE TYPE txn_status AS ENUM ('pending', 'processing', 'completed', 'failed',
'reversed', 'disputed');
CREATE TYPE txn_type AS ENUM (
    'deposit', 'withdrawal', 'transfer_in', 'transfer_out',
    'payment', 'refund', 'fee', 'interest', 'adjustment',
    'ach_credit', 'ach_debit', 'wire_in', 'wire_out',
    'card_purchase', 'card_refund', 'atm_withdrawal', 'atm_deposit'
);
CREATE TYPE entry_side AS ENUM ('debit', 'credit');
CREATE TYPE card_status AS ENUM ('ordered', 'active', 'blocked', 'expired',
'cancelled');
CREATE TYPE alert_severity AS ENUM ('low', 'medium', 'high', 'critical');

-- ============================================================
-- CUSTOMERS (KYC)
-- ============================================================
CREATE TABLE customers (
    customer_id     UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    -- PII stored encrypted in production
    email           VARCHAR(320)    NOT NULL,
    phone           VARCHAR(30),
    full_name       VARCHAR(200)    NOT NULL,
    date_of_birth   DATE,
    tax_id_hash     BYTEA,          -- pgcrypto encrypted SSN/TIN
    nationality     CHAR(2),
    country_of_residence CHAR(2),
    status          customer_status NOT NULL DEFAULT 'prospect',
    kyc_status      kyc_status      NOT NULL DEFAULT 'not_started',
    kyc_reviewed_at TIMESTAMPTZ,
    kyc_reviewer    UUID,
    risk_score      NUMERIC(5,2)    DEFAULT 0,   -- AML risk score
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_customers_email UNIQUE (email)
);

CREATE TABLE customer_addresses (
    address_id      UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    customer_id     UUID            NOT NULL REFERENCES customers(customer_id),
    address_type    VARCHAR(30)     NOT NULL DEFAULT 'residential',
    line1           VARCHAR(255)    NOT NULL,
    line2           VARCHAR(255),
    city            VARCHAR(100)    NOT NULL,
    state           VARCHAR(100),
    postal_code     VARCHAR(20)     NOT NULL,
```

```
    country_code    CHAR(2)         NOT NULL,
    valid_from      DATE            NOT NULL DEFAULT CURRENT_DATE,
    valid_until     DATE,
    is_current      BOOLEAN         NOT NULL DEFAULT TRUE
);

CREATE TABLE kyc_documents (
    doc_id          UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    customer_id     UUID            NOT NULL REFERENCES customers(customer_id),
    doc_type        VARCHAR(50)     NOT NULL,   --
'passport','drivers_license','utility_bill'
    doc_number_hash BYTEA,          -- encrypted
    issuing_country CHAR(2),
    issue_date      DATE,
    expiry_date     DATE,
    storage_ref     VARCHAR(500),   -- reference to encrypted document store
    verified        BOOLEAN         NOT NULL DEFAULT FALSE,
    verified_at     TIMESTAMPTZ,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- ============================================================
-- ACCOUNTS
-- ============================================================
CREATE TABLE accounts (
    account_id      UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    account_number  VARCHAR(30)     NOT NULL,   -- formatted display number
    iban            VARCHAR(34),
    customer_id     UUID            NOT NULL REFERENCES customers(customer_id),
    account_type    account_type    NOT NULL,
    status          account_status  NOT NULL DEFAULT 'active',
    currency        currency        NOT NULL DEFAULT 'USD',
    -- Balance maintained by triggers/application; NEVER edited directly
    current_balance NUMERIC(19,4)   NOT NULL DEFAULT 0,
    available_balance NUMERIC(19,4) NOT NULL DEFAULT 0,   -- current - holds
    overdraft_limit NUMERIC(19,4)   NOT NULL DEFAULT 0,
    interest_rate   NUMERIC(7,4)    NOT NULL DEFAULT 0,
    credit_limit    NUMERIC(19,4),  -- for credit accounts
    opened_at       TIMESTAMPTZ     NOT NULL DEFAULT now(),
    closed_at       TIMESTAMPTZ,
    last_txn_at     TIMESTAMPTZ,
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_account_number UNIQUE (account_number),
    CONSTRAINT chk_balance_overdraft
        CHECK (current_balance >= -overdraft_limit),
    CONSTRAINT chk_credit_limit
        CHECK (credit_limit IS NULL OR current_balance >= -credit_limit)
);

-- Balance history for point-in-time queries (regulatory requirement)
CREATE TABLE account_balance_snapshots (
    snapshot_id     UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    account_id      UUID            NOT NULL REFERENCES accounts(account_id),
    balance         NUMERIC(19,4)   NOT NULL,
    available_balance NUMERIC(19,4) NOT NULL,
    snapshot_date   DATE            NOT NULL,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT uq_balance_snapshot UNIQUE (account_id, snapshot_date)
);

-- ============================================================
-- TRANSACTIONS (IMMUTABLE)
-- ============================================================
-- Transactions are IMMUTABLE once completed.
-- Corrections are made via reversal transactions, never UPDATE.

CREATE TABLE transactions (
    txn_id          UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    txn_reference   VARCHAR(50)     NOT NULL,   -- unique business reference
    txn_type        txn_type        NOT NULL,
    status          txn_status      NOT NULL DEFAULT 'pending',
    initiated_by    UUID            REFERENCES customers(customer_id),
    amount          NUMERIC(19,4)   NOT NULL,
    currency        currency        NOT NULL,
    description     VARCHAR(500),
    merchant_name   VARCHAR(200),
    merchant_mcc    CHAR(4),        -- Merchant Category Code
    channel         VARCHAR(50),    -- 'online','mobile','branch','atm','api'
    reference_txn_id UUID           REFERENCES transactions(txn_id),  -- for
reversals
    idempotency_key VARCHAR(100),
    initiated_at    TIMESTAMPTZ     NOT NULL DEFAULT now(),
    processed_at    TIMESTAMPTZ,
    value_date      DATE,
    metadata        JSONB,
    CONSTRAINT uq_txn_reference UNIQUE (txn_reference),
    CONSTRAINT uq_idempotency UNIQUE (idempotency_key),
    CONSTRAINT chk_amount_positive CHECK (amount > 0)
) PARTITION BY RANGE (initiated_at);

CREATE TABLE transactions_2024_q1 PARTITION OF transactions
    FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
CREATE TABLE transactions_2024_q2 PARTITION OF transactions
    FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
CREATE TABLE transactions_2024_q3 PARTITION OF transactions
    FOR VALUES FROM ('2024-07-01') TO ('2024-10-01');
CREATE TABLE transactions_2024_q4 PARTITION OF transactions
    FOR VALUES FROM ('2024-10-01') TO ('2025-01-01');

-- ============================================================
-- DOUBLE-ENTRY LEDGER
-- ============================================================
-- The ledger is the source of truth for all account balances.
-- For every transaction, there must be at least one debit entry
```

```sql
-- and one credit entry where SUM(debits) == SUM(credits).

CREATE TABLE ledger_entries (
    entry_id        UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    txn_id          UUID            NOT NULL REFERENCES transactions(txn_id),
    account_id      UUID            NOT NULL REFERENCES accounts(account_id),
    side            entry_side      NOT NULL,
    amount          NUMERIC(19,4)   NOT NULL,
    currency        currency        NOT NULL,
    running_balance NUMERIC(19,4)   NOT NULL,   -- account balance after this entry
    entry_date      DATE            NOT NULL DEFAULT CURRENT_DATE,
    value_date      DATE,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT chk_entry_amount CHECK (amount > 0)
) PARTITION BY RANGE (entry_date);

-- Invariant check: debit sum == credit sum per transaction
-- Enforced by application + periodic reconciliation job

-- ============================================================
-- HOLDS (PENDING CHARGES)
-- ============================================================
CREATE TABLE holds (
    hold_id         UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    account_id      UUID            NOT NULL REFERENCES accounts(account_id),
    txn_id          UUID            REFERENCES transactions(txn_id),
    amount          NUMERIC(19,4)   NOT NULL,
    currency        currency        NOT NULL,
    reason          VARCHAR(200),
    expires_at      TIMESTAMPTZ     NOT NULL,
    released_at     TIMESTAMPTZ,
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- ============================================================
-- TRANSFERS
-- ============================================================
CREATE TABLE transfers (
    transfer_id     UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    txn_id          UUID            NOT NULL REFERENCES transactions(txn_id),
    from_account_id UUID            NOT NULL REFERENCES accounts(account_id),
    to_account_id   UUID            REFERENCES accounts(account_id),  -- NULL for
external
    to_external_account VARCHAR(50),  -- IBAN or account number for external
    amount          NUMERIC(19,4)   NOT NULL,
    currency        currency        NOT NULL,
    fx_rate         NUMERIC(12,6),  -- for cross-currency transfers
    to_amount       NUMERIC(19,4),  -- amount in destination currency
    transfer_type   VARCHAR(30)     NOT NULL,   -- 'internal','ach','wire','sepa'
    routing_number  VARCHAR(20),
    swift_code      VARCHAR(12),
```

```
    initiated_at    TIMESTAMPTZ     NOT NULL DEFAULT now(),
    settled_at      TIMESTAMPTZ,
    CONSTRAINT chk_transfer_dest CHECK (
        (to_account_id IS NOT NULL) != (to_external_account IS NOT NULL)
    )
);

-- ============================================================
-- CARDS
-- ============================================================
CREATE TABLE cards (
    card_id         UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    account_id      UUID            NOT NULL REFERENCES accounts(account_id),
    card_type       VARCHAR(20)     NOT NULL,   -- 'debit','credit','prepaid'
    -- PAN stored encrypted / tokenized (never store raw PAN)
    pan_token       VARCHAR(500)    NOT NULL,
    last_four       CHAR(4)         NOT NULL,
    expiry_month    SMALLINT        NOT NULL,
    expiry_year     SMALLINT        NOT NULL,
    network         VARCHAR(20)     NOT NULL,   -- 'VISA','MASTERCARD','AMEX'
    status          card_status     NOT NULL DEFAULT 'ordered',
    daily_limit     NUMERIC(12,2),
    contactless     BOOLEAN         NOT NULL DEFAULT TRUE,
    issued_at       TIMESTAMPTZ,
    activated_at    TIMESTAMPTZ,
    cancelled_at    TIMESTAMPTZ,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- ============================================================
-- STANDING ORDERS & DIRECT DEBITS
-- ============================================================
CREATE TABLE standing_orders (
    so_id           UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    account_id      UUID            NOT NULL REFERENCES accounts(account_id),
    to_account_id   UUID            REFERENCES accounts(account_id),
    to_external_account VARCHAR(50),
    amount          NUMERIC(19,4)   NOT NULL,
    currency        currency        NOT NULL,
    frequency       VARCHAR(20)     NOT NULL,   --
'weekly','monthly','quarterly','annual'
    start_date      DATE            NOT NULL,
    end_date        DATE,
    next_execution  DATE            NOT NULL,
    last_executed   DATE,
    description     VARCHAR(300),
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- ============================================================
-- FRAUD DETECTION
```

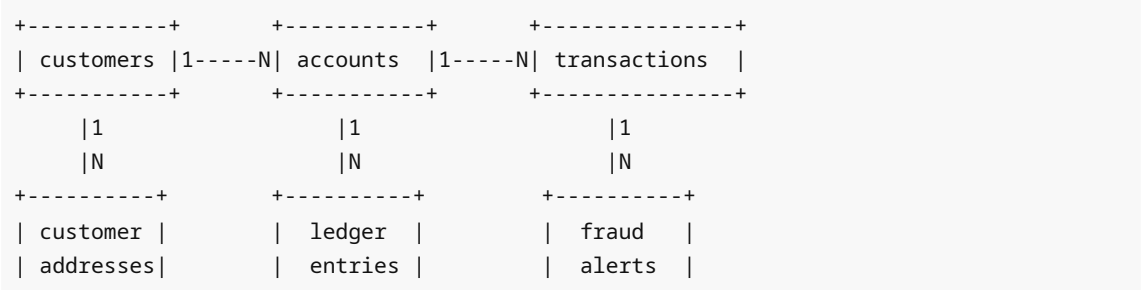
```sql
-- ============================================================
CREATE TABLE fraud_rules (
    rule_id         SERIAL          PRIMARY KEY,
    name            VARCHAR(200)    NOT NULL,
    description     TEXT,
    rule_type       VARCHAR(50)     NOT NULL,   --
'velocity','geo','amount','pattern'
    config          JSONB           NOT NULL,   -- rule parameters
    severity        alert_severity  NOT NULL,
    is_active       BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

CREATE TABLE fraud_alerts (
    alert_id        UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    txn_id          UUID            NOT NULL REFERENCES transactions(txn_id),
    account_id      UUID            NOT NULL REFERENCES accounts(account_id),
    rule_id         INT             REFERENCES fraud_rules(rule_id),
    severity        alert_severity  NOT NULL,
    score           NUMERIC(5,2)    NOT NULL,   -- 0-100 risk score
    reason          TEXT,
    metadata        JSONB,          -- rule-specific context
    status          VARCHAR(30)     NOT NULL DEFAULT 'open',  --
'open','investigating','closed','false_positive'
    investigated_by UUID            REFERENCES customers(customer_id),
    resolved_at     TIMESTAMPTZ,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- Velocity counters (rolling window aggregates for fraud rules)
CREATE TABLE fraud_velocity (
    account_id      UUID            NOT NULL REFERENCES accounts(account_id),
    window_type     VARCHAR(20)     NOT NULL,   -- '1h','24h','7d','30d'
    txn_count       INT             NOT NULL DEFAULT 0,
    total_amount    NUMERIC(19,4)   NOT NULL DEFAULT 0,
    last_countries  TEXT[],         -- last 5 transaction countries
    updated_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    PRIMARY KEY (account_id, window_type)
);

-- ============================================================
-- REGULATORY REPORTING
-- ============================================================
CREATE TABLE aml_reports (
    report_id       UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    customer_id     UUID            NOT NULL REFERENCES customers(customer_id),
    report_type     VARCHAR(50)     NOT NULL,   -- 'SAR','CTR','STR'
    reporting_period DATE,
    status          VARCHAR(30)     NOT NULL DEFAULT 'draft',
    submitted_at    TIMESTAMPTZ,
    regulatory_ref  VARCHAR(100),
    report_data     JSONB,
```

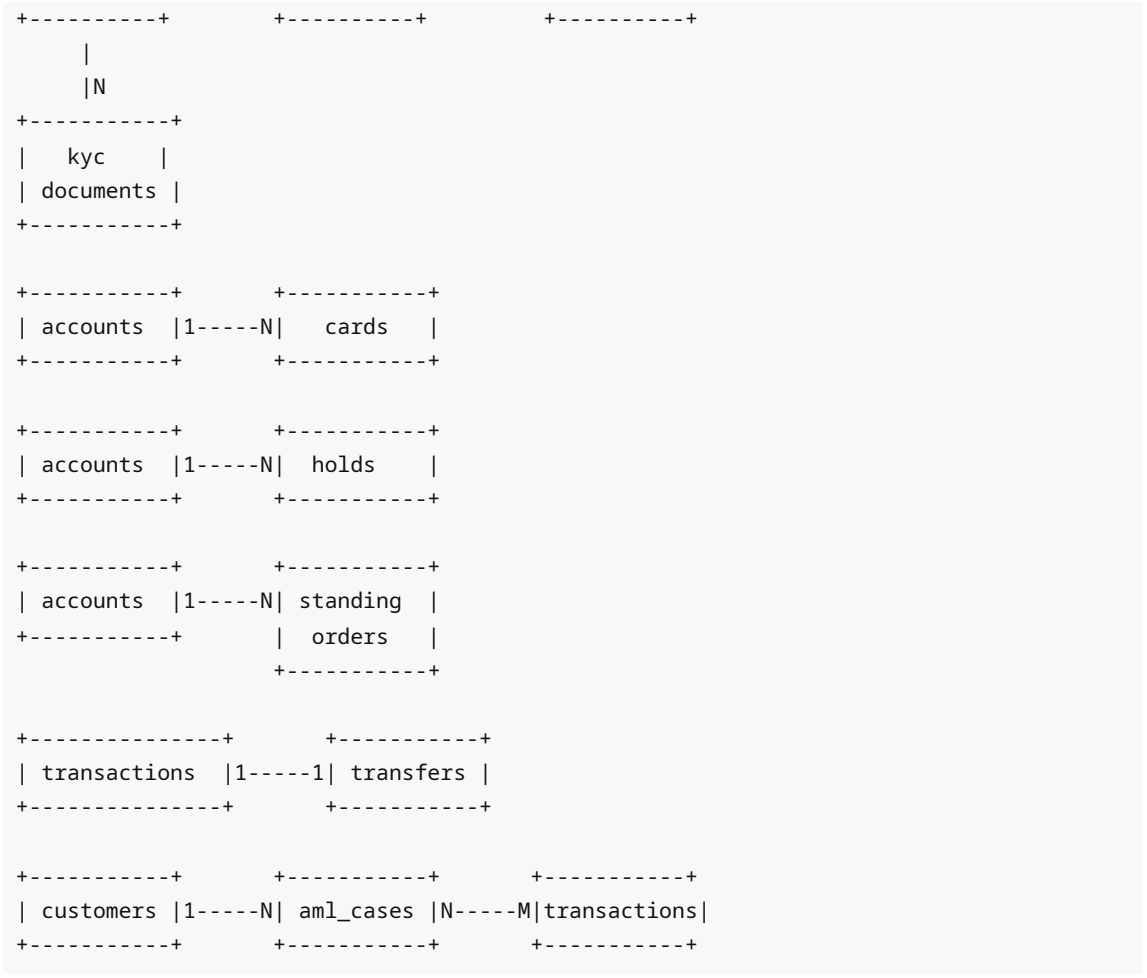
```
    created_by      UUID,
    created_at      TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- Suspicious transaction groupings
CREATE TABLE aml_cases (
    case_id         UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    customer_id     UUID            NOT NULL REFERENCES customers(customer_id),
    case_type       VARCHAR(50)     NOT NULL,
    risk_level      alert_severity  NOT NULL,
    status          VARCHAR(30)     NOT NULL DEFAULT 'open',
    opened_at       TIMESTAMPTZ     NOT NULL DEFAULT now(),
    closed_at       TIMESTAMPTZ,
    assigned_to     UUID,
    notes           TEXT
);

CREATE TABLE aml_case_transactions (
    case_id         UUID            NOT NULL REFERENCES aml_cases(case_id),
    txn_id          UUID            NOT NULL REFERENCES transactions(txn_id),
    added_at        TIMESTAMPTZ     NOT NULL DEFAULT now(),
    PRIMARY KEY (case_id, txn_id)
);

-- ============================================================
-- AUDIT LOG (IMMUTABLE)
-- ============================================================
CREATE TABLE audit_log (
    log_id          UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    table_name      VARCHAR(100)    NOT NULL,
    record_id       UUID            NOT NULL,
    operation       CHAR(1)         NOT NULL,   -- 'I','U','D'
    changed_by      UUID,
    changed_at      TIMESTAMPTZ     NOT NULL DEFAULT now(),
    old_values      JSONB,
    new_values      JSONB,
    ip_address      INET,
    session_id      VARCHAR(100)
) PARTITION BY RANGE (changed_at);

-- Audit trigger factory
CREATE OR REPLACE FUNCTION audit_trigger_function()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO audit_log (table_name, record_id, operation, changed_by, old_values,
new_values)
    VALUES (
        TG_TABLE_NAME,
        CASE WHEN TG_OP = 'DELETE' THEN OLD.customer_id ELSE NEW.customer_id END,
        LEFT(TG_OP, 1),
        current_setting('app.current_user_id', TRUE)::UUID,
        CASE WHEN TG_OP != 'INSERT' THEN row_to_json(OLD)::JSONB END,
```

```
        CASE WHEN TG_OP != 'DELETE' THEN row_to_json(NEW)::JSONB END
    );
    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER audit_customers
    AFTER INSERT OR UPDATE OR DELETE ON customers
    FOR EACH ROW EXECUTE FUNCTION audit_trigger_function();
-- (Similar triggers on accounts, cards, standing_orders)

-- ============================================================
-- BALANCE UPDATE FUNCTION (Core Business Logic)
-- ============================================================
CREATE OR REPLACE FUNCTION process_transfer(
    p_from_account_id   UUID,
    p_to_account_id     UUID,
    p_amount            NUMERIC,
    p_currency          currency,
    p_description       VARCHAR,
    p_idempotency_key   VARCHAR
) RETURNS UUID AS $$
DECLARE
    v_txn_id    UUID;
    v_bal_from  NUMERIC;
    v_ref       VARCHAR;
BEGIN
    -- Idempotency check
    SELECT txn_id INTO v_txn_id
    FROM transactions WHERE idempotency_key = p_idempotency_key;
    IF FOUND THEN RETURN v_txn_id; END IF;

    -- Lock accounts in consistent order to prevent deadlock
    SELECT current_balance INTO v_bal_from
    FROM accounts
    WHERE account_id = p_from_account_id
    FOR UPDATE;

    -- PERFORM for the target account lock
    PERFORM current_balance
    FROM accounts
    WHERE account_id = p_to_account_id
    FOR UPDATE;

    -- Sufficient funds check
    IF v_bal_from < p_amount THEN
        RAISE EXCEPTION 'insufficient_funds' USING ERRCODE = 'P0001';
    END IF;

    -- Generate reference
    v_ref := 'TXN' || TO_CHAR(now(), 'YYYYMMDD') || substr(uuid_generate_v4()::text,
1, 8);
```

```
    -- Create transaction record
    INSERT INTO transactions (txn_id, txn_reference, txn_type, status, amount,
currency, description, idempotency_key, processed_at)
    VALUES (uuid_generate_v4(), v_ref, 'transfer_out', 'completed', p_amount,
p_currency, p_description, p_idempotency_key, now())
    RETURNING txn_id INTO v_txn_id;

    -- Debit source account
    UPDATE accounts
    SET current_balance = current_balance - p_amount,
        available_balance = available_balance - p_amount,
        last_txn_at = now(),
        updated_at = now()
    WHERE account_id = p_from_account_id;

    INSERT INTO ledger_entries (txn_id, account_id, side, amount, currency,
running_balance, entry_date)
    SELECT v_txn_id, p_from_account_id, 'debit', p_amount, p_currency,
           current_balance, CURRENT_DATE
    FROM accounts WHERE account_id = p_from_account_id;

    -- Credit destination account
    UPDATE accounts
    SET current_balance = current_balance + p_amount,
        available_balance = available_balance + p_amount,
        last_txn_at = now(),
        updated_at = now()
    WHERE account_id = p_to_account_id;

    INSERT INTO ledger_entries (txn_id, account_id, side, amount, currency,
running_balance, entry_date)
    SELECT v_txn_id, p_to_account_id, 'credit', p_amount, p_currency,
           current_balance, CURRENT_DATE
    FROM accounts WHERE account_id = p_to_account_id;

    RETURN v_txn_id;
END;
$$ LANGUAGE plpgsql;
```

ASCII ER Diagram





Indexing Strategy

```sql
-- Customers: authentication and KYC lookups
CREATE INDEX idx_customers_email        ON customers (lower(email));
CREATE INDEX idx_customers_kyc          ON customers (kyc_status) WHERE kyc_status
!= 'approved';
CREATE INDEX idx_customers_risk         ON customers (risk_score DESC) WHERE
risk_score > 50;

-- Accounts: customer portfolio lookup
CREATE INDEX idx_accounts_customer      ON accounts (customer_id, account_type);
CREATE INDEX idx_accounts_number        ON accounts (account_number);
CREATE INDEX idx_accounts_iban          ON accounts (iban) WHERE iban IS NOT NULL;

-- Transactions: history by account (via ledger)
CREATE INDEX idx_transactions_initiated ON transactions (initiated_at DESC);
CREATE INDEX idx_transactions_status    ON transactions (status, initiated_at)
    WHERE status IN ('pending', 'processing');
CREATE INDEX idx_transactions_ref       ON transactions (txn_reference);
CREATE INDEX idx_transactions_idem      ON transactions (idempotency_key);
```

```
-- Ledger: account statement (critical hot path)
CREATE INDEX idx_ledger_account_date    ON ledger_entries (account_id, entry_date
DESC);
CREATE INDEX idx_ledger_txn             ON ledger_entries (txn_id);

-- Fraud: open alerts requiring review
CREATE INDEX idx_fraud_open             ON fraud_alerts (status, severity,
created_at DESC)
    WHERE status = 'open';
CREATE INDEX idx_fraud_account          ON fraud_alerts (account_id, created_at
DESC);

-- Holds: active holds per account (impacts available_balance)
CREATE INDEX idx_holds_active           ON holds (account_id, expires_at)
    WHERE is_active = TRUE;

-- Audit log: compliance queries
CREATE INDEX idx_audit_record           ON audit_log (table_name, record_id,
changed_at DESC);
CREATE INDEX idx_audit_user             ON audit_log (changed_by, changed_at DESC)
    WHERE changed_by IS NOT NULL;

-- AML cases
CREATE INDEX idx_aml_customer           ON aml_cases (customer_id, status);
CREATE INDEX idx_aml_open               ON aml_cases (status, risk_level)
    WHERE status = 'open';
```

Query Patterns

Q1 — Account Statement (most common read)

```
SELECT
    le.entry_id,
    le.entry_date,
    le.value_date,
    t.txn_reference,
    t.txn_type,
    t.description,
    t.merchant_name,
    le.side,
    le.amount,
    le.currency,
    le.running_balance
FROM ledger_entries le
JOIN transactions t ON t.txn_id = le.txn_id
WHERE le.account_id = $1
  AND le.entry_date BETWEEN $2 AND $3
ORDER BY le.entry_date DESC, le.created_at DESC
LIMIT 100 OFFSET $4;
```

Q2 — Transfer Funds (via stored function)

```
-- Idempotent: safe to retry
SELECT process_transfer(
    p_from_account_id  => $1::UUID,
    p_to_account_id    => $2::UUID,
    p_amount           => $3::NUMERIC,
    p_currency         => $4::currency,
    p_description      => $5,
    p_idempotency_key  => $6
);
```

Q3 — Customer Full Portfolio

```
SELECT
    a.account_id,
    a.account_number,
    a.account_type,
    a.status,
    a.currency,
    a.current_balance,
    a.available_balance,
    a.interest_rate,
    a.credit_limit,
    a.last_txn_at,
    (SELECT COUNT(*) FROM cards c WHERE c.account_id = a.account_id AND c.status =
'active') AS active_cards
FROM accounts a
WHERE a.customer_id = $1
  AND a.status != 'closed'
ORDER BY a.opened_at;
```

Q4 — Fraud Velocity Check (called before every transaction)

```
-- Check 1h velocity
SELECT txn_count, total_amount
FROM fraud_velocity
WHERE account_id = $1 AND window_type = '1h';

-- Also check recent unique countries
SELECT DISTINCT json_array_elements_text(
    (SELECT last_countries::JSON FROM fraud_velocity WHERE account_id = $1 AND
window_type = '24h')
) AS country;
```

Q5 — Daily Ledger Reconciliation (ensures debit = credit per transaction)

```sql
SELECT
    t.txn_id,
    t.txn_reference,
    SUM(CASE WHEN le.side = 'debit' THEN le.amount ELSE 0 END) AS total_debits,
    SUM(CASE WHEN le.side = 'credit' THEN le.amount ELSE 0 END) AS total_credits,
    SUM(CASE WHEN le.side = 'debit' THEN le.amount ELSE 0 END) -
    SUM(CASE WHEN le.side = 'credit' THEN le.amount ELSE 0 END) AS imbalance
FROM transactions t
JOIN ledger_entries le ON le.txn_id = t.txn_id
WHERE t.initiated_at::date = CURRENT_DATE - 1
  AND t.status = 'completed'
GROUP BY t.txn_id, t.txn_reference
HAVING ABS(
    SUM(CASE WHEN le.side = 'debit' THEN le.amount ELSE 0 END) -
    SUM(CASE WHEN le.side = 'credit' THEN le.amount ELSE 0 END)
) > 0.001;  -- should return 0 rows; any row is a critical alert
```

Q6 — AML: High-Value Transaction Report (CTR — transactions > $10K)

```sql
SELECT
    c.customer_id, c.full_name,
    a.account_number,
    t.txn_id, t.txn_reference,
    t.txn_type, t.amount, t.currency,
    t.initiated_at,
    t.merchant_name
FROM transactions t
JOIN ledger_entries le ON le.txn_id = t.txn_id
JOIN accounts a ON a.account_id = le.account_id
JOIN customers c ON c.customer_id = a.customer_id
WHERE t.initiated_at::date = $1   -- report date
  AND t.amount >= 10000
  AND t.status = 'completed'
ORDER BY t.amount DESC;
```

Q7 — Standing Orders Due Today

```sql
SELECT
    so.so_id, so.account_id, so.to_account_id,
    so.to_external_account, so.amount, so.currency,
    so.description
FROM standing_orders so
WHERE so.next_execution <= CURRENT_DATE
  AND so.is_active
  AND (so.end_date IS NULL OR so.end_date >= CURRENT_DATE)
ORDER BY so.next_execution;
```

Q8 — Account Balance at Historical Date (Point-in-Time)

```sql
-- First try snapshots (daily)
SELECT balance, available_balance, snapshot_date
FROM account_balance_snapshots
WHERE account_id = $1
  AND snapshot_date <= $2
ORDER BY snapshot_date DESC
LIMIT 1;

-- If no snapshot, compute from ledger (slower, used for gaps)
SELECT running_balance
FROM ledger_entries
WHERE account_id = $1
  AND entry_date <= $2
ORDER BY entry_date DESC, created_at DESC
LIMIT 1;
```

Q9 — Open Fraud Alerts Dashboard

```sql
SELECT
    fa.alert_id, fa.severity, fa.score, fa.reason,
    fa.created_at,
    a.account_number,
    c.full_name AS customer_name,
    t.txn_reference, t.amount, t.currency, t.txn_type
FROM fraud_alerts fa
JOIN accounts a ON a.account_id = fa.account_id
JOIN customers c ON c.customer_id = a.customer_id
JOIN transactions t ON t.txn_id = fa.txn_id
WHERE fa.status = 'open'
ORDER BY fa.severity DESC, fa.score DESC, fa.created_at ASC
LIMIT 50;
```

Q10 — Interest Calculation for Savings Accounts

```sql
SELECT
    a.account_id,
    a.account_number,
    a.current_balance,
    a.interest_rate,
    ROUND(
        a.current_balance * a.interest_rate / 365 * $days,
        4
    ) AS interest_amount,
    a.currency
FROM accounts a
WHERE a.account_type = 'savings'
  AND a.status = 'active'
  AND a.current_balance > 0;
```

Scaling Strategy

Correctness First, Then Scale

Banking systems prioritize correctness. Only introduce horizontal scaling after exhausting vertical options and functional decomposition.

Phase 1: Single Primary (0–5M customers)

- Single PostgreSQL with synchronous replica (synchronous_commit = on)
- All operations at SERIALIZABLE isolation for transfers
- READ COMMITTED acceptable for reporting queries
- PgBouncer for connection pooling (transaction mode)

Phase 2: Functional Decomposition (5M–20M customers)

- Separate Ledger DB from CRM DB (customers, KYC) from Fraud DB
- Ledger DB: partitioned by quarter, archival after 2 years
- Reporting replica (REPEATABLE READ isolation) for analytics
- Kafka: TransactionCompleted events → fraud service → AML service

Phase 3: Regional (20M+ customers)

- Multi-region read replicas for account balance reads
- All writes to primary (consistency requirement)
- Saga pattern for multi-step transfers across services
- Two-phase commit (2PC) or compensating transactions for distributed transfers
- Consider: Citus for horizontal sharding by customer_id

Isolation Levels by Operation

Operation	Isolation Level	Reason
Transfer funds	SERIALIZABLE	Prevent phantom reads causing double-spend
Balance inquiry	READ COMMITTED	Snapshot of current committed balance
Statement generation	REPEATABLE READ	Consistent view across many rows
Fraud rule check	READ COMMITTED	Acceptable slight lag; speed matters
Audit report	REPEATABLE READ	Consistent snapshot for compliance

---

Tradeoffs

Decision	Chosen	Alternative	Why
Immutable transactions	No UPDATE on completed txns	Mutable with history table	Simplifies audit; corrections use reversal entries (standard accounting practice)
Double-entry ledger	Two entries per transaction	Single-entry balance update	Enables reconciliation, audit, and fraud detection. Debit == Credit is mathematically verifiable

Balance stored on account	Denormalized current_balance	Always compute from ledger	Reduces aggregation latency for balance checks; must be kept in sync by transfers stored procedure
SERIALIZABLE for transfers	SERIALIZABLE	REPEATABLE READ + explicit lock	SERIALIZABLE catches more anomalies; PostgreSQL's SSI has low overhead
Encrypted PII	pgcrypto in-column	Application-layer encryption	Encryption closer to data; harder to accidentally expose in logs. Application-layer gives more control

---

Interview Discussion Points

1. **Why double-entry bookkeeping?** Every debit has an equal and opposite credit. This means the sum of all account balances in the system should always equal zero (assets = liabilities). This is a self-validating invariant — if the ledger is imbalanced, a bug exists. It is the foundation of banking for 500+ years.
2. **How do you prevent double-spend?** The `process_transfer` function acquires row-level locks on both accounts in a deterministic order (always lock the lower account_id first to prevent deadlock), checks the balance, and commits atomically. The idempotency_key ensures retried API calls don't re-execute.
3. **Why SERIALIZABLE for transfers?** REPEATABLE READ can still allow serialization anomalies (write skew). For example, two concurrent overdraft checks might both see a sufficient balance and both proceed, resulting in a combined overdraft. SERIALIZABLE prevents this.
4. **How is the audit log maintained?** A trigger on every sensitive table (`customers` , `accounts` , `cards`) fires after every INSERT/UPDATE/DELETE and writes the old and new row values (as JSONB) to `audit_log` . The `app.current_user_id` session variable is set at connection time by the application.

Common Interview Follow-ups

Q: How do you handle a failed inter-bank transfer? A: Use a saga pattern. Step 1: Debit source account (write debit ledger entry). Step 2: Send to ACH network. If Step 2 fails, a compensating transaction credits the source account back. Never leave money in a "deducted but not sent" limbo — the reversal creates a balancing credit entry.

Q: How do you detect money laundering? A: Structuring detection: flag accounts with multiple transactions just under $10K within 24 hours (CTR threshold). Network analysis: identify accounts that receive money from many sources and immediately transfer out (smurfing). The `fraud_velocity` table tracks rolling window counts and amounts to enable fast rule evaluation.

Q: How do you handle regulatory data archival? A: Transactions are partitioned by quarter. After 7 years (regulatory minimum), old partitions are `DETACH` ed and exported to immutable cold storage (S3 with Object Lock). The ledger table references partition-local data. Compliance queries can reattach partitions if needed.

Performance Considerations

- **synchronous_commit = on**: Required for banking. Never turn this off for writes involving money.
- **fsync = on**: Must never be disabled. Data loss on crash is unacceptable.
- **Partition pruning on transactions**: Always include `initiated_at` in WHERE clauses for time-range queries to trigger partition pruning.
- **HOT updates on accounts**: `current_balance` is updated on every transaction. With `fillfactor=80`, PostgreSQL can do HOT updates in-place without touching the index.
- **Avoid SELECT FOR UPDATE across replicas**: Locking is only valid on the primary. Direct all write-path reads to the primary.
- **Batch standing orders**: Process standing_orders in batches of 1000 using a loop, not all at once, to avoid holding locks for too long.

Cross-References

- See `01_amazon_marketplace_db.md` for payment processing integration patterns
- See `21_System_Design/03_banking_system_design.md` for full system architecture
- See `21_System_Design/08_system_design_framework.md` for interview framework
- PostgreSQL isolation levels: Chapter 6 of this guide
- ACID transactions: Chapter 4 of this guide
- Table partitioning: Chapter 5 of this guide